SCE Working Group 1 · Project A · October 2026

Semantic ontologies

Primer for SCE-WG1

Matt Mackay & Akshay Shanker

Communicating with AI

AI. AI can now write code, solve hard math problems and translate papers to code.

Humans. We need a common structure to understand *what* AI "understands":

1. **Agreement on the *meaning*** of a paper, a model or a model description.
2. **Inspect and verify** AI outputs.
3. Efficiently **scale and expand** the set of problems AI can solve.

Semantics ≠ Text

Semantics. Meaning is a relation between a piece of text and some thing (domain) outside language.

Missing meaning. A system trained only on text cannot contain the relationship of the text to meaning (Bender and Koller 2020).

A large language model fills in the meaning statistically, without any explicit inspectable structure.

Bender and Koller (2020), "Climbing towards NLU", ACL.

Semantic Ontology

Meaning of objects is not enough.

Ontology. To "understand", we need a structure of relations between objects of different *types*

Semantic Ontology. Machine-readable account of objects, their types and the valid relations between them.

Signatures state the argument types of relations and the input and output types of operations.

Three parts to a semantic ontology

$$\text{written form} \xrightarrow{\ \Upsilon\ } \text{domain of meaning}$$

1. The written form. Machine-readable symbols, each with a type and a signature ($v : X \to \mathbb{R}$).

2. The domain of meaning. The objects we model, such as state spaces, laws, value functions and operators, are sorted into kinds with relations between them.

3. The map Y (upsilon). It sends a symbol of a given type to an object of the matching kind, and a program checks that it does.

> Example from AI training. With NVIDIA's TensorFloat-32 (TF32) mode, a matrix of 32-bit floating-point numbers (`float32`) can change even when multiplied by the identity, because inputs are rounded first. A shared numerical ontology makes the written form (type and arithmetic settings), domain of meaning (machine values and rounded operations) and Y (the interpretation map) explicit, giving humans and AI tools the same arithmetic assumptions to follow.

Harel and Rumpe (2004), "Meaningful modeling: what's the semantics of 'semantics'?", *Computer* 37(10), 64–72, for the three parts · Bellman calculus, Introduction §1 (bellman-ddsl/docs) · in this deck a type belongs to the written form and a kind to the domain; Guizzardi and Guarino (2024) write "type" for what the deck calls a kind. · PyTorch, "Numerical accuracy", TF32 section · Stosic and Micikevicius (NVIDIA, 2021), "Accelerating AI Training with NVIDIA TF32 Tensor Cores".

Ways to represent and use an ontology

Formal mathematics is one way to express an ontology.

FORM OR USE	WHAT IT MAKES EXPLICIT	EXAMPLES IN THIS DECK
Typed mathematical theory, including algebra	Types, operation signatures and laws.	Math-in-the-Middle; Bellman calculus.
Ontology axioms	Formal statements taken as given about the domain, for example “Every household is an economic agent.”	OntoPlan, using the ontology language OWL.
Graph representation	Object identifiers, declared types and relations.	OntoPlan’s scene graph, represented in RDF.
Annotations to models or data	Links from variables or objects to ontology terms.	SemGen models; NVIDIA’s scene objects.

> These forms can be combined: an OWL ontology can be represented as an RDF graph, and annotations can also be RDF statements. In every case, Y interprets the written symbols in the domain of meaning.

Kohlhase et al. (2017), §§2–3 · W3C: OWL 2 overview and RDF graph model; OWL 2 Primer, §§3, 4.2 (axioms and inference) · Nam et al. (2026), §3.1 · Neal et al. (2019), §2 · NVIDIA, semantic labels. RDF represents statements as subject–relation–object triples. The household example is illustrative.

The scope depends on the field and the task

Domains of meaning vary by field. Examples include genes and biological processes, objects and actions in robotics, and economic models, methods and data.

One domain can support several ontologies. In computational economics, one ontology may describe model assumptions for comparison; another may classify numerical methods for solver selection.

For an economics project. An ontology can specify value functions and valid compositions of operators; define firm and worker types and the employment relations allowed in a graph; or link a dataset column to a shared definition of annual consumption per household and its units.

> WG1's task. Define the scope for computational economics: which objects, relations and operations should our ontologies describe, and which tasks should they support?

On choosing an ontology's domain, purpose and scope: Noy and McGuinness (2001), *Ontology Development 101*, §3, Step 1. The computational-economics uses above are illustrative.

Semantic ontologies and formal methods in AI

USE	PUBLISHED FINDING	SOURCE
AI training data (NVIDIA)	NVIDIA labels Omniverse objects with Wikidata identifiers through OpenUSD's semantic-label schema. The labels supply object categories for AI training data.	NVIDIA specification OpenUSD schema
Robot planning (OntoPlan)	OntoPlan achieved **89% task success**, versus **27%** for SayPlan, with the same scene facts.	Nam et al. (2026) NeurIPS (accepted)
Robot experience (KnowRob)	KnowRob used an ontology to retrieve grasp positions and forces from recorded robot actions.	Beetz et al. (2018) ICRA
AI program translation	Retraining AI to emit numerical expressions for software evaluation raised C-to-assembly translation test accuracy from **3.8% to 92.6% on 21,104 functions**.	Zhang et al. (2024) EMNLP Findings
AI theorem proving (LeanDojo)	Retrieving existing Lean library results raised proof success from **23.2% to 26.3% on 2,000 theorems** requiring results not used in training.	Yang et al. (2023) NeurIPS
Numerical checking (SMT-LIB)	SMT-LIB defines numerical types, operations and rounding modes that reasoning software uses to check floating-point computations.	Brain et al. (2015) IEEE ARITH

Reported AI gains concern the combined systems described in each paper.

Evidence: shared models and operational uses

USE	PUBLISHED FINDING	SOURCE
Factory planning	Software converted an existing factory ontology and machine-operation descriptions into **267 planning actions in about 15 seconds**.	Malburg et al. (2023) Eng. Appl. AI
Assembly planning (Ford)	Ford's deployed assembly-planning AI used ontologies to interpret work instructions and share knowledge across applications.	Rychtyckyj et al. (2016) IAAI
Materials data (MDS-Onto)	MDS-Onto defines terms for X-ray and solar-energy measurements; software uses these terms to generate data templates and machine-readable datasets.	Rajamohan et al. (2025) Scientific Data
Model composition (SemGen)	SemGen used ontology terms to match variables by biological meaning and combine models in different formats.	Neal et al. (2019) Bioinformatics
Arterial-model assembly	Software used ontology annotations and conservation rules to assemble an **86-vessel arterial model from three templates**.	Shahidi et al. (2021) PLOS Comput. Biol.
Biological inference (GO-CAM)	Gene Ontology uses logical rules to infer additional labels for gene functions and distributes them in standard releases.	Thomas et al. (2019) Nature Genetics
Mathematical software	A shared mathematical ontology let SageMath, GAP and Singular perform different steps of one polynomial calculation.	Kohlhase et al. (2017) MACIS

Factory conversion reuses an existing ontology; arterial-model assembly combines semantic annotations with physical constraints.

Case studies

Case study 1 of 3

OntoPlan: robot task planning

Motivating example from robotics

Nam et al. (2026), accepted at NeurIPS 2026.

OntoPlan combines a language model with planning software for simulated indoor robot tasks. The ontology defines object types, relations and inference rules; a matching action model specifies what actions change.

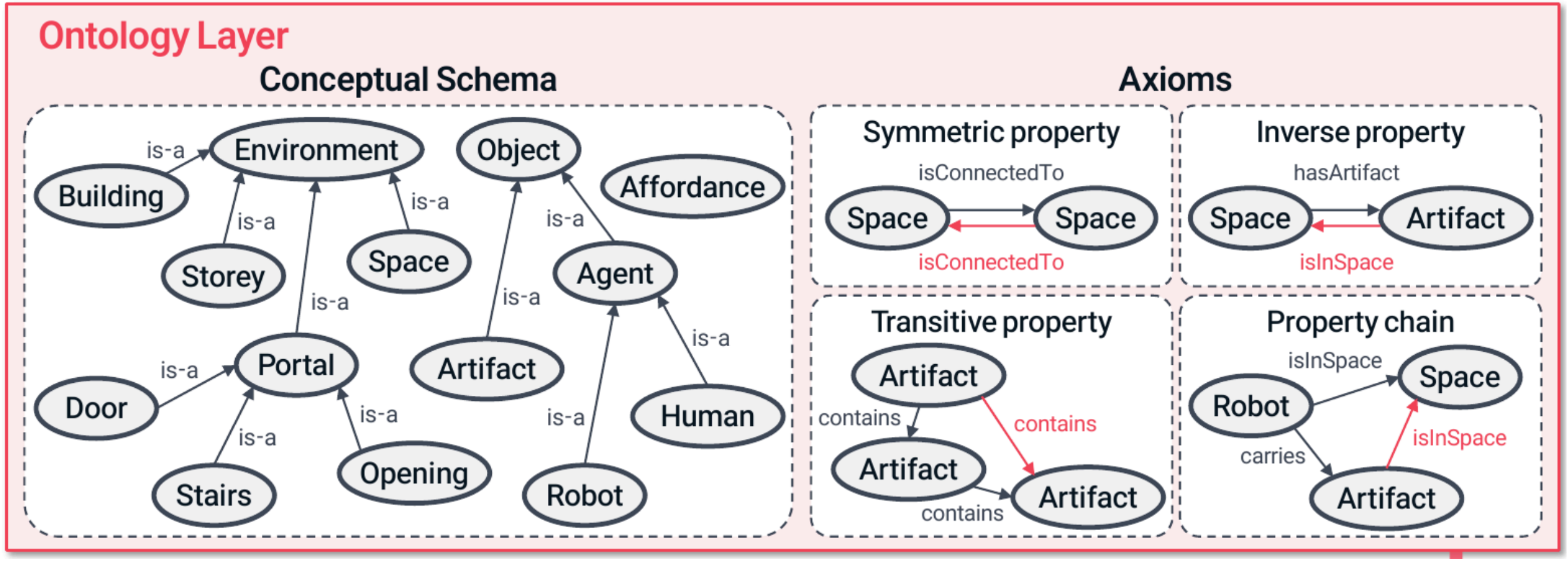
Illustrative task. Move a toothbrush from a cabinet to another room.

MODEL COMPONENT	TOOTHBRUSH EXAMPLE
Objects and types	The toothbrush and cabinet have type **Artifact** (physical object); the room has type **Space**.
Relation	The toothbrush is inside the cabinet.
Action effect	After pickup, the toothbrush is held by the robot's hand and is no longer inside the cabinet.
Inference rule	A carried object is in the same room as the robot.

> The AI and planning software use the same objects, types and updated relations.

Nam et al. (2026), OntoPlan, §3.1 and Appendix A, Table 7. The task illustrates the paper's state updates.

OntoPlan's ontology defines object types and rules



Written form. The diagram shows type names and logical rules. **Domain of meaning.** These describe the simulated rooms, robots, objects and relations. **Y.** It interprets `Space` as the set of spaces and `carries` as the relation between a robot and its carried objects.

Nam et al. (2026), OntoPlan, Figure 2, ontology layer (cropped). The full figure also shows rooms, objects and their states. Y is this deck's notation for interpretation.

Ontology-based planning achieved 89% task success

The same study compared three AI planning systems on 150 simulated robot tasks. All received the same scene facts and action rules and used GPT-4o.

SYSTEM	SUCCESS RATE
OntoPlan	**88.7% (133/150)**
SayPlan	27%
DELTA	13%

OntoPlan retrieves relevant facts, constructs plans and records the consequences of actions. DELTA already had formal action rules and software that constructs plans.

Nam et al. (2026), OntoPlan, §§4–5 and Appendix E. Results evaluate the ontology together with fact retrieval and planning software.

OntoPlan kept AI inputs small as scenes grew

Within OntoPlan, the authors compared selecting relevant facts with supplying all scene facts as text.

FACTS SUPPLIED TO AI	GROWTH IN THE LARGEST INPUT
Relevant facts selected from the ontology	**1.06×**
All scene facts supplied as text	**8.13×**

Task success was 89% with selected facts and 87% with all facts; the difference was not statistically significant.

Nam et al. (2026), OntoPlan, Table 6. Ratios compare large with small scenes; input size counts tokens (text units) in the largest single call per task.

Case study 2 of 3

Buffer-stock saving

Mathematical Description of a Model

We want to show that machine reading a mathematical description of a model misses meaning. Consider an excerpt of a written statement of the buffer-stock saving model:

> *Each period the household arrives with assets k_t, receives income y_t and earns the gross return $R\,k_t$ on those assets, where R is the gross interest factor. Market resources are then $m_t = R\,k_t + y_t$. Income is i.i.d. with $\log y_t \sim N(\mu_y, \sigma_y^2)$. The household chooses consumption $c_t \in (0, m_t]$, leaving $k_{t+1} = m_t - c_t$, to solve the Bellman equation*
>
> $$v_t(m_t) = \max_{0<c_t\le m_t} \Big\{ u(c_t) + \beta\,\mathbb{E}_t\, v_{t+1}\big(R\,(m_t - c_t) + y_{t+1}\big) \Big\}.$$
>
> *The right-hand side defines an operator $\mathbb{T}$ with $v_t = \mathbb{T}\,v_{t+1}$, which sends the function v_{t+1} to the function v_t, backward from a given terminal value over a finite horizon.*

Context

Implicit context Economist write code to implement $\mathbb{T}$ by using their implicit understanding of Dynamic Programming.

By itself, machine reading the statement does not give enough information to build $\mathbb{T}$.

- The challenge is fundamental, since a machine parses text into a syntax tree by explicit rules, whereas the tree, without added structure on its symbols, does not capture the meaning of the model. See the appendix on parse trees.
- A semantic ontology makes that implicit understanding explicit.

Missing Meaning Problem

A non-exhaustive list of meanings that are missed in parsing the text describing the buffer-stock saving model.

MISSING MEANING	IN THE BUFFER-STOCK STATEMENT
Variable bindings	The maximization binds c_t, while the expectation binds y_{t+1}. The left-hand side binds m_t by defining v_t as a function of m_t, which leaves m_t free inside the right-hand side.
Parameters	The paper defines u, R, β and the law of y_{t+1} elsewhere, in no fixed place. Only β and R are numbers, u is a function and the law is a measure - this is not immediately apparent from the statement.
Symbol types	The value function v_t maps $\mathbb{R}_{++}$ to $\mathbb{R}$, and m_t a point of in its domain. The tree records both symbols but not the fact that $v_t(m_t)$ is a function evaluation at a variable of a particular type.
An equation between functions	For each m_t the Bellman equation equates two numbers. Over all m_t it is one equation between functions, $v_t = \mathbb{T}\, v_{t+1}$, in which $\mathbb{T}$ maps v_{t+1} to v_t.

Missing Meaning Problem

MISSING MEANING	IN THE BUFFER-STOCK STATEMENT
Whether a symbol is a variable, a function, or a random variable	The symbol m_t, defined by $m_t = R\,k_t + y_t$, is i) the path of one household's resources, ii) the function $(k, y) \mapsto Rk + y$ composed inside the operator, and iii) the random variable whose law is the cross-sectional distribution. A paper writes one symbol for all three meanings, whereas code must represent them as three objects of different types "doing" different things.
Shock law	The statement fixes $\log y_t \sim N(\mu_y, \sigma_y^2)$. In the shorter form $y \sim \text{LogNormal}(\mu_y, \sigma_y)$, μ_y denotes the mean of $\log y$ under one convention and the mean of y under another.

The LogNormal convention: Project Bellman (the author's project, slide 23), reference page "Shock distributions".

Missing Meaning Problem

MISSING MEANING	IN THE BUFFER-STOCK STATEMENT
Implications of timing stated in words	"A policy shock raises entrants at date t." If the entrants change y_t in equilibrium, the sentence is consistent with a model in which prices respond at t and with one in which they respond only at $t+1$.
What the household knows when it decides	If the return R_t is random, persistent and drawn before the decision, the household's expectation of R_{t+1} depends on whether it observes R_t or only $m_t = R_t\, k_t + y_t$ when it decides.

Multiple possible parsed meanings

Several readings. One well-formed statement admits two or more readings of its parse tree. The appendix gives examples of false agreement.

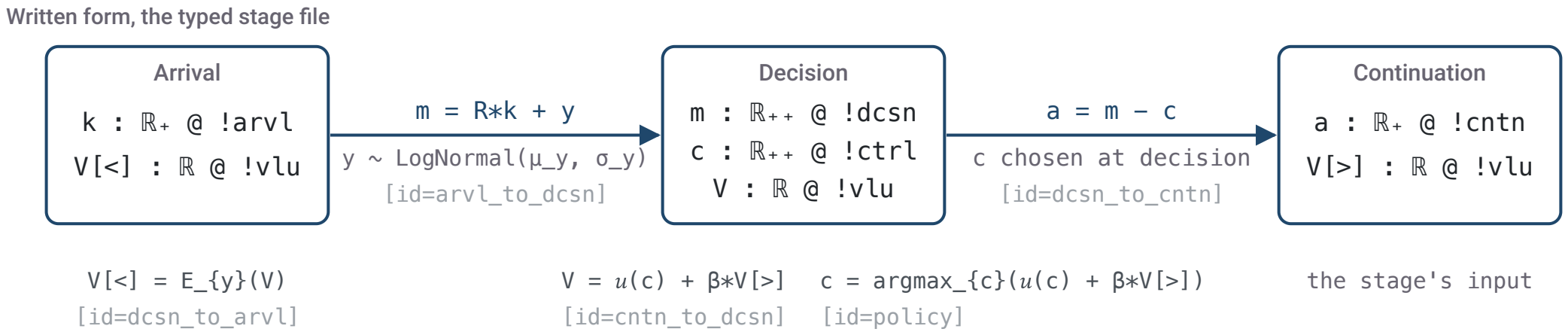
Human readers. False agreement rarely arises when an economist reads the model. An economist has been taught that m_t in $v(m_t)$ is the point at which v is evaluated, not a path of assets across a population.

Timing and information. Even so, the timing of events, and what the individual knows at each decision, confuse economists themselves.

AI readers. When we rely on an AI's statistical reading of the text, we can never be sure that it understands the model correctly, even if it appears to most of the time.

The ontology: one stage of the buffer-stock model

The Bellman-SYM stage file is the **written form**; its **domain of meaning** contains state spaces, probability laws, value functions and operators. **Y** maps each declaration and block to the mathematical object it denotes.



Perches. The three information sets of the period: arrival `!arvl`, decision `!dcsn` and continuation `!cntn`.

Fields. A state or a value declared at a perch with its set, as in `m : ℝ₊₊ @ !dcsn`.

Blocks. The equations between two perches: the transition with the shock, the choice, and the expectation.

Bellman-SYM, Stages §§1–4 (the documentation example, in the letters k, m, a of the calculus); Bellman calculus, Stages and perches §1 (bellman-ddsl/docs) · the law line fixes the convention that slide 20 left to a subfield.

What the ontology checks and makes explicit

Y (upsilon) reads each field and block of the file as a typed object. The elaborator, the program that constructs those objects, refuses a line that the types do not admit and names the rule.

THE FILE WRITES	UNDER Y IT DENOTES
`V[>] : ℝ @ !vlu` at the continuation perch	$v_{\succ} : \mathbb{R}_+ \to \mathbb{R}$, a function of savings, never a number
the choice block `[id=cntn_to_dcsn]` with `policy`	$\mathbb{T}_{\succ} : \mathsf{V}_{\succ} \to \mathsf{V}$, the choice operator
the expectation block `[id=dcsn_to_arvl]`	$\mathbb{T}_{\prec} : \mathsf{V} \to \mathsf{V}_{\prec}$, the expectation operator

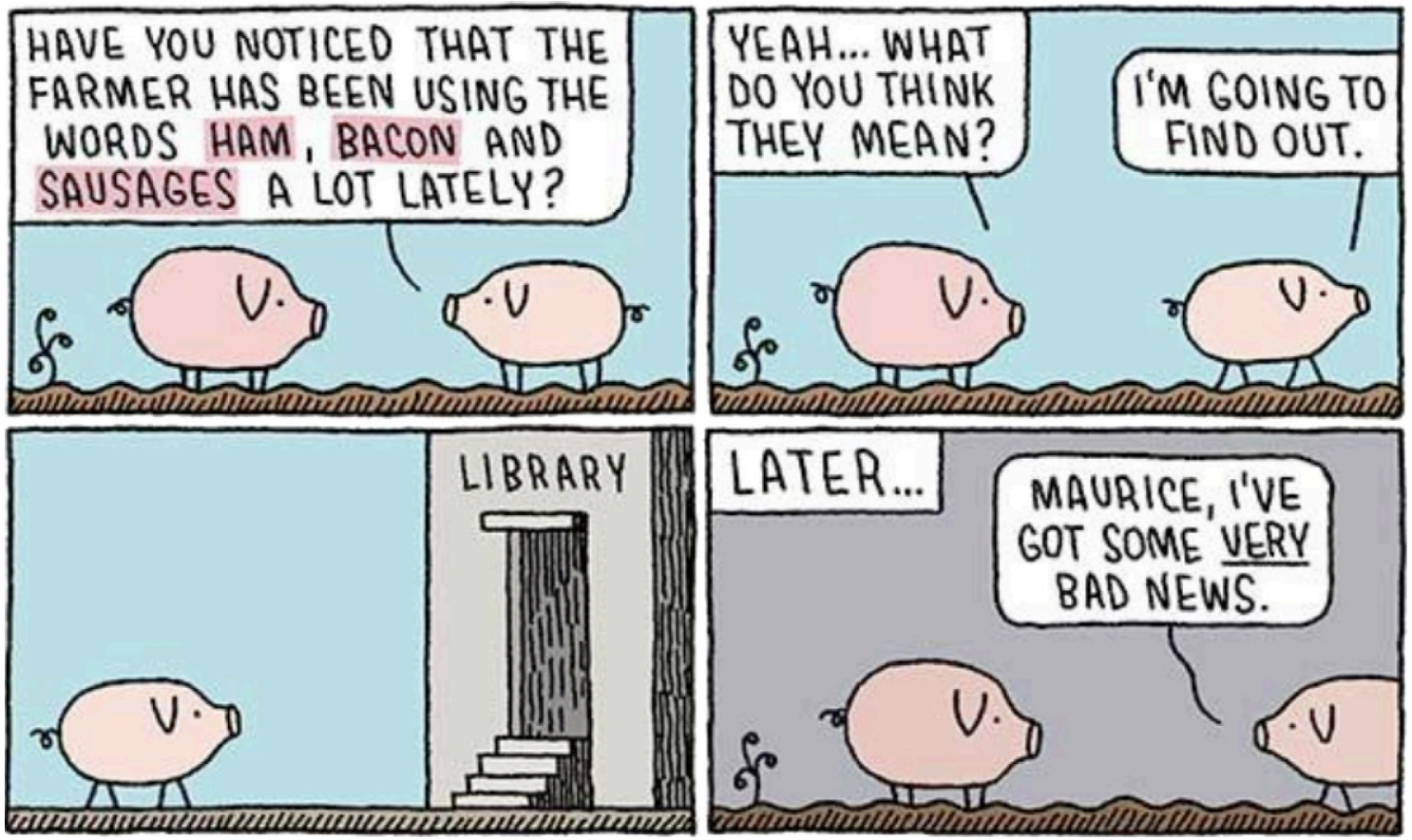
A LINE THAT	THE ELABORATOR
reads the shock beside its expectation, `V[<] = E_{y}(V) + y`	refuses it under `shock-reintroduced`, since y is not known at arrival
lists a continuation value at the decision perch	refuses it under `field-perch`, since the value is a function of savings
puts the mean of the shock in the transition, `m = R*k + exp(μ_y + σ_y^2/2)`	accepts it, and the different model stands visible on that line
writes a second `β` in the expectation block	accepts it, and the second `β` stands visible on its own line
would impose a stationary closure $v_{t+1} = v_t$	has no line for it, since `V[>]` comes from the next period or the terminal boundary

The reference on refusals, rules shock-reintroduced and field-perch; Bellman-SYM, Field equations §5, Perch tagging rules §3, Periods §§2–3, Trellises §2; the runs of `elaborate_text` of 13 October 2026.

Case study 3 of 3

Pig goes to the library

What the pig learned in the library



© Tom Gauld, reproduced in Guizzardi and Guarino (2024), "Explanation, semantics, and ontology", *Data & Knowledge Engineering* 153, 102325, Figure 1; shown here for discussion within the working group.

The set-theoretic semantics is not enough

What must a reader know to understand the joke?

Set-theoretic semantics. A function from each word to a subset of a domain D of individuals gives $[\![ham]\!],\ [\![bacon]\!],\ [\![sausages]\!] \subseteq D$. Such a mapping, the paper says, "would not do Maurice here any good".

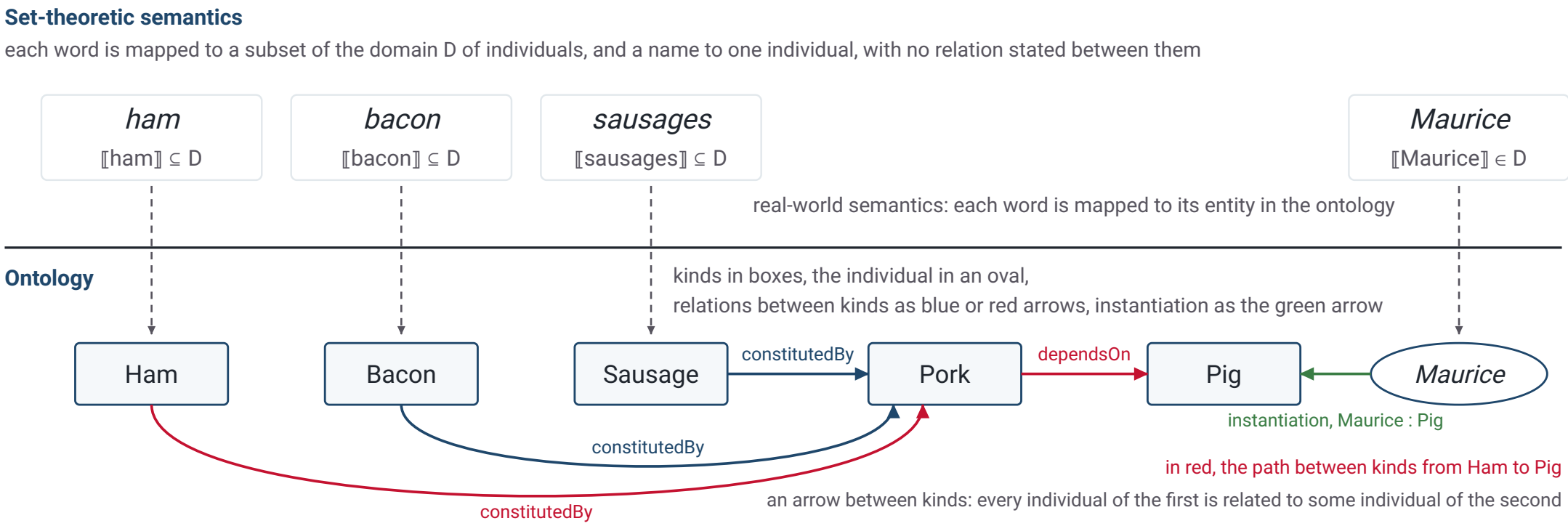
Ontology. The kinds and relations the words are about form a separate system, which the paper states as three facts. Ham, bacon and sausage are food products constituted by pork. Pork comes from a pig. Maurice is a pig.

- In the deck's notation: $Maurice : \mathsf{Pig},\ \mathsf{Ham} \sqsubseteq \exists constitutedBy.\mathsf{Pork}$ (every ham is constituted by some pork), $\mathsf{Pork} \sqsubseteq \exists dependsOn.\mathsf{Pig}$ (every quantity of pork comes from some pig).
- The library holds this. The dictionary does not.

The joke here arises from the system of kinds and relations, not from the set-theoretic semantics.

Guizzardi and Guarino (2024), "Explanation, semantics, and ontology", *Data & Knowledge Engineering* 153, 102325, §2.1: the quotation, and the three facts (1)–(3) that understanding the cartoon needs, stated there in words; the symbols are the deck's.

When Maurice learns the ontology



The words above are the **written form**; the kinds, individuals and relations represented below form the **domain of meaning**. The dashed arrows show **Y**, which assigns each word its meaning; the solid arrows state relations without identifying Maurice as the source of any particular ham.

Guizzardi and Guarino (2024), §2.1: "Mapping the words 'ham', 'bacon' and 'sausages' to subsets of a semantic domain would not do Maurice here any good"; the three facts (1)–(3); real-world semantics "maps symbols to the entities (types, relations, objects, events, aspects) that are part of this ontology".

Semantics and Ontology

Set. A set-theoretic semantics maps each word to a subset of a domain D of individuals, $[\![ham]\!] \subseteq D$, and nothing more.

Type. The ontology is a separate system of kinds, individuals and relations, written as typed symbols.

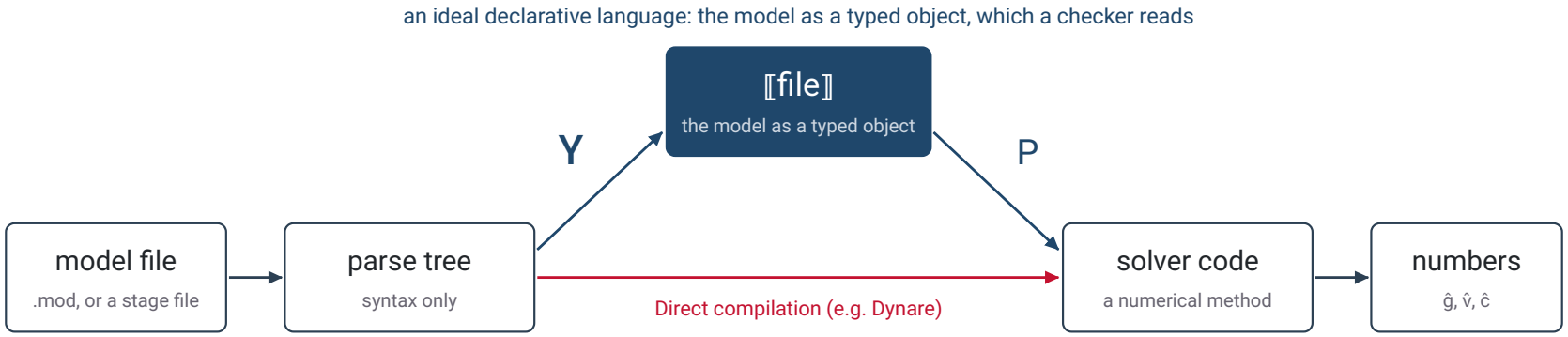
- Kinds: Pig, Pork, Ham, Bacon, Sausage. The word ham names the kind Ham, whose instances are the individual hams.
- Individual: $Maurice$: Pig. A typed symbol pairs a symbol with the kind of what it names. Here the colon is the instantiation relation, which holds between the individual Maurice and the kind Pig.
- Relations: $constitutedBy$, a relation from Ham to Pork, and $dependsOn$, a relation from Pork to Pig. Each holds between individuals of its two kinds and is not a function from one kind to the other. The kind-level facts Ham $\sqsubseteq \exists constitutedBy.$Pork and Pork $\sqsubseteq \exists dependsOn.$Pig say that every ham is constituted by some pork and that every quantity of pork comes from some pig.

	THE PIGS	THE BUFFER-STOCK MODEL
Written form	ham, $bacon$, $sausages$	m, y, $\mathbb{E}$, v, $\mathbb{T}$ in the Bellman equation
Semantics, $[\![\cdot]\!]$ into a set	$[\![ham]\!]$, $[\![bacon]\!]$, $[\![sausages]\!] \subseteq D$	$[\![\text{Bellman equation}]\!] = \mathbb{T}$, some operator
Left open	foods for pigs, or foods from pigs	which variable $\max$ chooses and which $\mathbb{E}$ averages over, the space of v, the law and timing of y
Ontology, typed statements	$Maurice$: Pig, Ham $\sqsubseteq \exists constitutedBy.$Pork, Pork $\sqsubseteq \exists dependsOn.$Pig	$v : \mathbb{R}_{++} \to \mathbb{R}$, $y : \mathbb{R}_{++}$ with law ν_y, $\mathbb{T} : (\mathbb{R}_{++} \to \mathbb{R}) \to (\mathbb{R}_{++} \to \mathbb{R})$
False agreement	two pigs agree that the words name foods, one meaning foods for pigs and the other foods from pigs	two readers accept the Bellman equation and disagree on the timing of the price response to a shock

Guizzardi and Guarino (2024), "Explanation, semantics, and ontology", *Data & Knowledge Engineering* 153, 102325, §2.1 (semantics as a function; an ontology as "a specific theory about the kinds of entity and their ties"; types, individuals and instantiation, where the paper's word for a kind is "type"; the facts (1) and (2) are stated there in words and written above in the deck's notation), §3.4 (false agreement).

From code to numerical results

Both routes parse a model file and produce numerical code. The upper route first states its economic interpretation: Y (upsilon) maps the written model to a typed model object, ⟦file⟧; P (rho) attaches a numerical method to it.



Beyond compilation. An explicit Y makes the model's economic objects and relations available for checking, separately from its numerical implementation.

Conceptual routes in this deck's notation: Y denotes a separately stated, shared economic interpretation. Compilers can already perform semantic checks; Dynare represents equations separately from solution commands. Clang documentation · Dynare manual · Bellman calculus, Introduction §1.

What AI needs beyond code compilation

Successful compilation or execution does not establish agreement with the intended economic model.

Shared definitions. Give the AI an ontology that specifies the economic objects, their units, timing and relations.

Links to code. Identify the variables and functions that implement each shared economic definition.

Checks on model agreement. Check declarations against the ontology, review the links to code, and test the numerical results.

> Example. Shared definitions distinguish assets chosen today from resources after tomorrow's income shock, even when both use floating-point numbers.

Y interprets the declared symbols as economic objects; reviewing their implementation is a separate task. Proposed requirements for AI model generation and translation. Precedent for shared definitions and mappings: Kohlhase et al. (2017), MACIS, §§2–5.

The community library

One model, several numerical implementations

The library asks each toolkit to implement a shared model and calibration, with its own solution method and runnable tutorial.

PART OF A CONTRIBUTION	BUFFER-STOCK EXAMPLE
Common economics	Equations, timing, shock laws, normalization and calibration.
Implementation choices	Solution method, shock approximation, grids, interpolation and software version.
Comparison quantities	Consumption, target resources, MPCs and model conditions.

> The starting point is the buffer-stock description and HARK tutorial in PR #5, under review.

Community Library · Buffer-stock submission guide. The result tables, metadata format and tolerances are optional draft conventions; reporting requirements remain under discussion. Status checked 15 October 2026.

Link toolkit variables to shared model definitions

PUBLISHED PRECEDENT	WHAT WAS DONE
SemGen	Ontology annotations matched biological quantities across different model formats.
Math-in-the-Middle	Symbols from SageMath, GAP and Singular were mapped to shared mathematical definitions.
BioModels	Curators annotated model entities and reproduced published simulations.

Proposed for the library. Link each implementation's variables and operations to agreed economic definitions. These links align written terms; **Y interprets the terms as economic objects.**

Neal et al. (2019), *Bioinformatics*, §§2–5 · Kohlhase et al. (2017), MACIS, §§2–5 · Malik-Sheriff et al. (2020), *Nucleic Acids Research*. The library extension is proposed; the papers establish the precedents.

Define the quantity before comparing the numbers

In the buffer-stock draft, **target resources** are where expected next-period resources equal current resources, measured relative to permanent income.

POSITIVE VALUES IN EACH SHOCK APPROXIMATION	SAVED HARK ESTIMATE OF TARGET RESOURCES
7	**1.3910**
21	**1.3994**

The same model and calibration give different numerical estimates when the shock approximation changes.

> Proposed extension. Each result would declare its model version, quantity definition, normalization and numerical settings. Reviewers would check these declarations against the code and numerical results.

Saved HARK sensitivity study, §3 · Quantity definition, §1. Both calculations retain the same additional zero-income outcome and asset grid. These are saved estimates, not a new computation.

Use the library to test WG1's common definitions

Proposed pilot. Start with the full buffer-stock description and HARK contribution, then add a second toolkit.

Task. Ask an AI to identify comparable results and report differences in numerical settings.

Comparison. Compare the AI's answers with and without semantic annotations, using the same complete model statement, equations, calibration and run information. Record correct matches, incorrect matches and cases requiring review.

> The library supplies the worked examples; the results help WG1 decide which definitions and checks to share.

Proposed pilot, not an existing library feature. The annotations restate supplied facts. The full buffer-stock model includes permanent and transitory shocks; the earlier Bellman stage illustration is simplified.

Appendix

Appendix

Further examples

Biology: the Gene Ontology

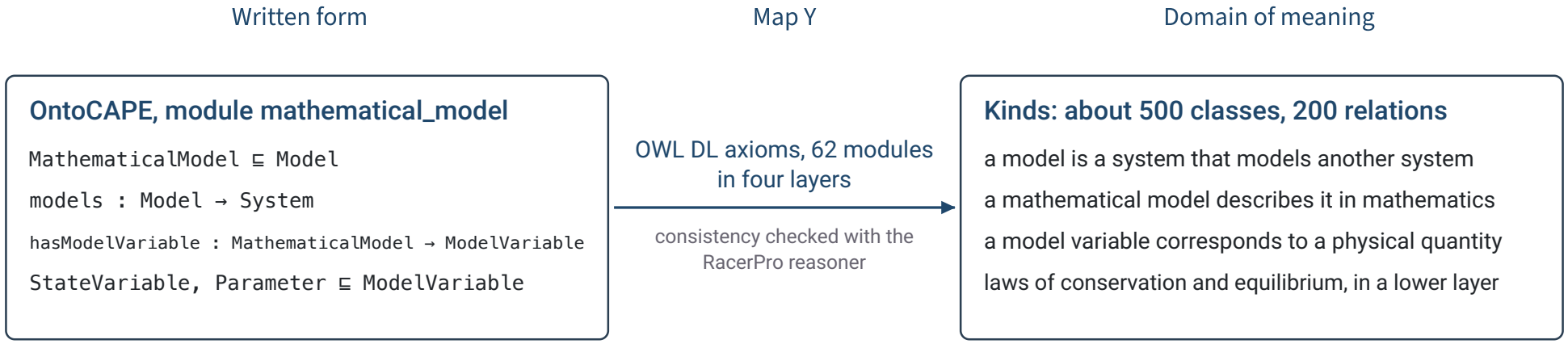


Excludes. A missing annotation read as the absence of a function (the open-world assumption), a NOT read as an inconclusive result, and a term read by its name rather than by its definition and relations, since a label must be unique and a definition must always denote the same referents (OBO Foundry principles 12 and 19).

For economics. A model's object is named by a term with a definition and relations, not by a variable name. The role tags `!ctrl` and `!vlu` and the law line of a stage file play that part, as the `sboTerm` of the Systems Biology Ontology does for a rate law in SBML.

Ashburner et al. (2000), "Gene Ontology: tool for the unification of biology", *Nature Genetics* 25(1), 25–29 · The Gene Ontology Consortium (2026), "The Gene Ontology knowledgebase in 2026", *Nucleic Acids Research* 54(D1), D1779–D1792, Table 3 (the release of 22 July 2025) · GAF 2.2 documentation and the human annotation file of 21 May 2026 (geneontology.org); the go-ontology build file, which runs the ELK reasoner · SBO: Courtot et al. (2011), *Molecular Systems Biology* 7, 543.

Engineering: OntoCAPE, the ontology for chemical process engineering



Excludes. A model confused with the system it models, and a model variable confused with the physical quantity it stands for: the relation `models` joins a model to its system, and `correspondsToQuantity` pairs a variable with a quantity, as a declared domain excludes the deck's function on the wrong space.

For economics. OntoCAPE is the nearest case to Project A, an ontology of models themselves: model variables, equation systems and solution strategies are kinds in its modules, so that a process model and its solver are distinct typed objects, as Y and P keep a model apart from its numerical method.

Morbach, Yang and Marquardt (2007), "OntoCAPE—A large-scale ontology for chemical process engineering", *Engineering Applications of Artificial Intelligence* 20(2), 147–161 · Morbach, Wiesner and Marquardt (2009), *Computers & Chemical Engineering* 33(10), 1546–1556 · Marquardt, Morbach, Wiesner and Yang (2010), *OntoCAPE: A Re-Usable Ontology for Chemical Process Engineering*, Springer · the OntoCAPE 2.0 distribution (AVT, RWTH Aachen, GPL): system.owl and mathematical_model.owl; technical reports LPT-2008-25 and LPT-2008-28; Morbach (2009), dissertation, RWTH Aachen, on the RacerPro checks.

Earth science: the SWEET ontologies

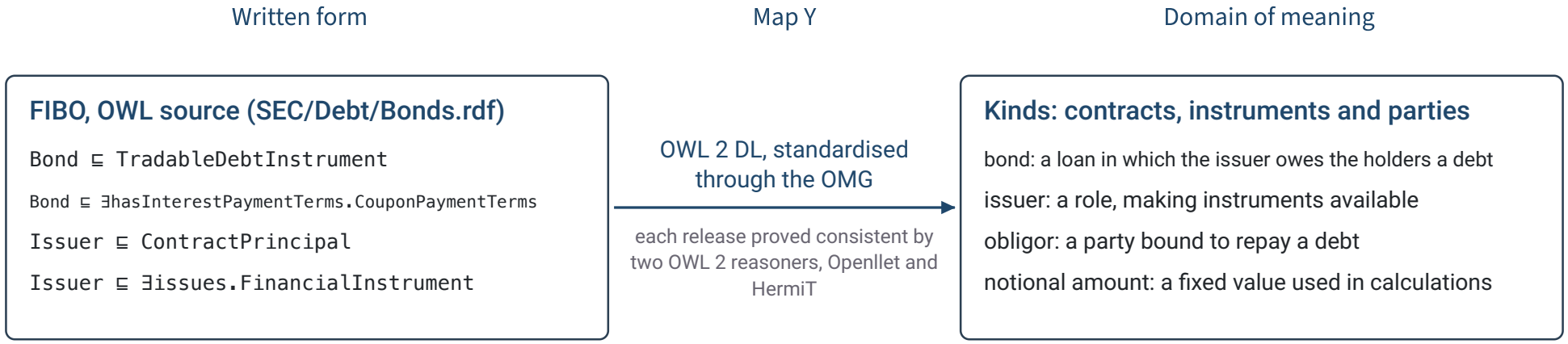


Excludes. A quantity name read as any of several kinds: a SWEET class fixes the kind by its definition and by its place among realms, quantities and phenomena. The CF standard names, published as SKOS concepts with their canonical units on the NERC vocabulary server, settle the same question for a netCDF variable such as `temp`, which is the law behind a parameter and timing stated in words in the deck's terms.

For economics. The comparison paper of the special issue that Project A proposes needs a vocabulary of this kind: which object each team reports, a value or a marginal value function, on which state and under which timing (den Haan, Judd and Juillard 2010, 2011).

Raskin and Pan (2005), "Knowledge representation in the semantic web for Earth and environmental terminology (SWEET)", *Computers & Geosciences* 31(9), 1119–1125 · ESIPFed/sweet, release 3.5.0 (14 July 2022), the header of sweetAll.ttl; the class count from the 3.5.0 source · CF standard names: NERC Vocabulary Server, collection P07, version 95 (16 September 2026), publisher Natural Environment Research Council, owner the CF Standard Names Committee · den Haan, Judd and Juillard (2010, 2011), *Journal of Economic Dynamics and Control* 34(1), 1–3 and 35(2), 175–177.

Finance: FIBO, the Financial Industry Business Ontology

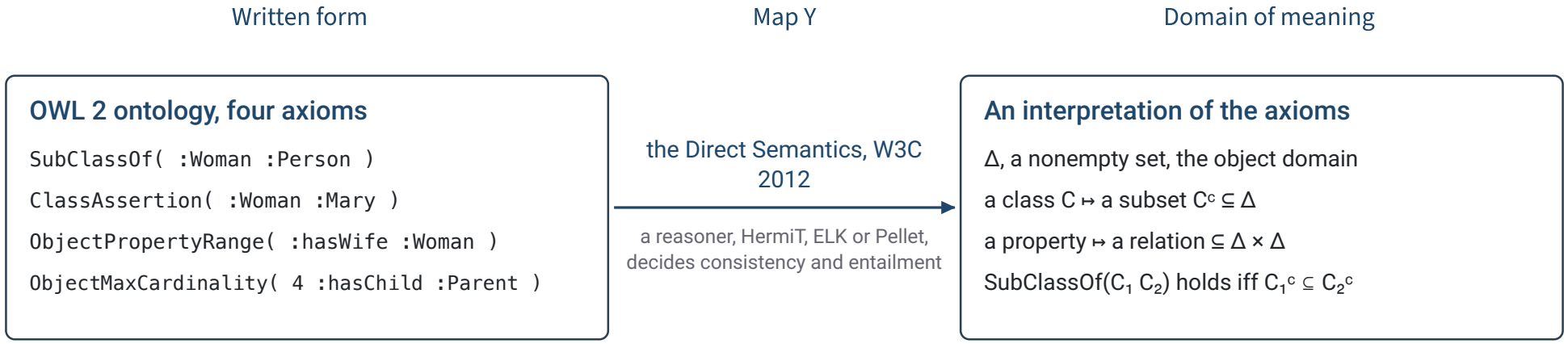


Excludes. An issuer read as the obligor of a bond, and a notional amount read as a principal: each is a defined class or property, and the 2023 revision of the Swaps ontology separated the nominal from the notional amount, which its change note says "were confused". In the deck's terms, a function on the wrong space and the law behind a parameter.

For economics. FIBO is finance's own ontology, with the Bureau of Economic Analysis, the Bureau of Labor Statistics, the CFTC and the Office of Financial Research among its contributors, and the 2026 rule under the Financial Data Transparency Act records it among the standards that commenters proposed. Project A's ontology names the objects of a model as FIBO names the parties and terms of a contract.

EDM Council, FIBO (spec.edmcouncil.org/fibo), release Q2 2026 (14 July 2026), MIT licence; the development-process page on consistency; onto-viewer's consistency checker (Openllet and HermiT) · OMG specifications EDMC-FIBO/FND 1.2, BE 1.1, FBC 1.0 and IND 1.0 (2017–2018) · Bennett (2013), "The financial industry business ontology: best practice for big data", *Journal of Banking Regulation* 14(3–4), 255–268 · Allemang, Garbacz, Grądzki, Kendall and Trypuz (2021), *FOIS 2021*, 112–126 · Financial Data Transparency Act Joint Data Standards, 91 FR 38246 (25 June 2026) · GoodRelations, the ontology schema.org absorbed in 2012: Hepp (2008), *EKAW 2008*, LNCS 5268, 329–346.

Knowledge representation: OWL 2, the language ontologies are written in



Excludes. Any reading of the axioms other than the one the Direct Semantics fixes: "the correct answer to any such question is predetermined by the formal semantics", whatever reasoner computes it (Primer §2). Nor is a missing statement read as false, or two names as two individuals.

For economics. This is the three-part definition in full: classes, properties and individuals as typed symbols, an interpretation as the domain, and a map whose consistency and entailments a reasoner decides. A stage file's Y plays the part of the Direct Semantics, and the elaborator the part of the reasoner.

W3C (2012), *OWL 2 Web Ontology Language*, Recommendations of 11 December 2012: Document Overview §2.3; Direct Semantics §§2.2–2.5; Profiles §§1 and 5; Primer §§2, 4 and 9 · Baader, Calvanese, McGuinness, Nardi and Patel-Schneider (eds.) (2007), *The Description Logic Handbook*, 2nd ed., Cambridge University Press · HermiT: Glimm, Horrocks, Motik, Stoilos and Wang (2014), *Journal of Automated Reasoning* 53(3), 245–269; ELK: Kazakov, Krötzsch and Simančík (2014), *Journal of Automated Reasoning* 53(1), 1–61.

The five reference ontologies against the three parts of a semantic ontology

Each row places one ontology against the three parts of slide 5, the typed symbols of the written form, the domain of meaning and the map between them, and names in the last column the program that checks the map.

ONTOLOGY	WRITTEN FORM	DOMAIN OF MEANING	MAP Y	CHECKER
Gene Ontology	the GO terms, each by ID	kinds of molecular function, biological process and cellular component	the term in a GAF annotation	the ELK reasoner on the ontology at every commit
OntoCAPE	OWL classes and relations in 62 modules	models, systems, materials and quantities	the axioms, `models` and `hasModelVariable`	the RacerPro reasoner
SWEET and the CF names	OWL classes, and SKOS concepts for the CF names	Earth-science quantities, realms and phenomena	the class, or the standard name on a variable	any OWL tool; the CF checker against the NVS table
FIBO	OWL classes and properties	contracts, instruments and parties	the class asserted of a record	two OWL 2 reasoners on every release
OWL 2	classes, properties and individuals	an interpretation: a set with relations on it	the Direct Semantics	a reasoner: consistency and entailment

Harel and Rumpe (2004), *Computer* 37(10), 64–72, for the three parts · the sources of each ontology are on its slide: Ashburner et al. (2000) and the Gene Ontology Consortium (2026); Morbach, Yang and Marquardt (2007); Raskin and Pan (2005) and the NERC Vocabulary Server; the EDM Council and Bennett (2013); W3C (2012).

The stage file against the same three parts

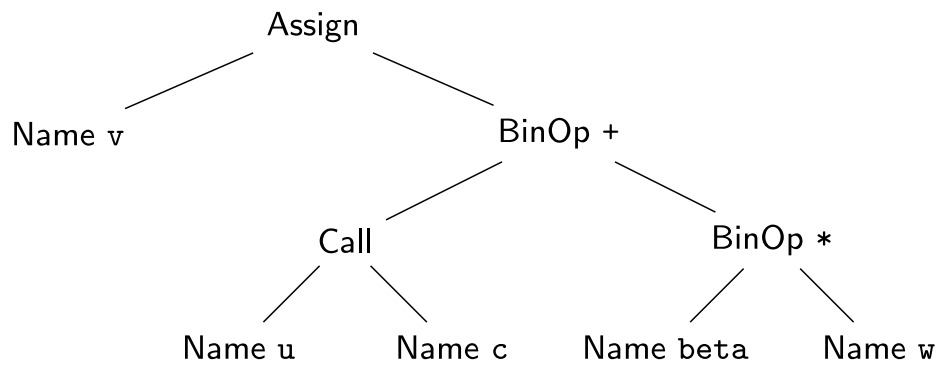
The stage file of slide 23, the deck's own written form, is read here against the columns of the table on slide 44.

PART	IN THE BELLMAN-SYM STAGE FILE
Written form	the fields and the blocks, each with a type and a perch, and the declared law of the shock (slide 23)
Domain of meaning	state spaces, laws, value functions and the operators between them (slide 5)
Map Y	the perch tags `@ !arvl` , `@ !dcsn` and `@ !cntn` , the role tags `!ctrl` and `!vlu` and the law line (slide 23), interpreted by Y (upsilon), the checked interpretation of slide 47, as the objects of slide 5
Checker	the elaborator (slide 24), which constructs the typed object or refuses the file, naming the rule the file breaks and the line; and the comparator, a program that compares two elaborated files up to renaming and reports, for each kind of item, agreement, difference or an undecided residue, the items it can neither match nor reject
Run on the file of slide 23	The elaborator constructed the object from the file and refused the line `E_{y}(V) + y` , with `y` listed among the block's inputs, under `shock-reintroduced` . The comparator read the file and the documentation example, with b and w for k and m, as one object under that renaming.

Bellman-SYM, Stages §5 (checked interpretation) · bellman-ddsl wiki, Y §1 (the elaborator) and judge §4 (the comparator) · the runs: `elaborate_text` of `bellman.factory.elaboration` and `compare` of `meaning_map_training.compare` , 13 October 2026, on the stage file of slide 23 and on Bellman-SYM's documentation example (Stages §4).

The parse tree of a written statement

Parsing a written statement into a tree, as every compiler first does, shows what the statement leaves to context.



What the parser does. It splits `v = u(c) + beta * w` into a tree by grammar rules alone.

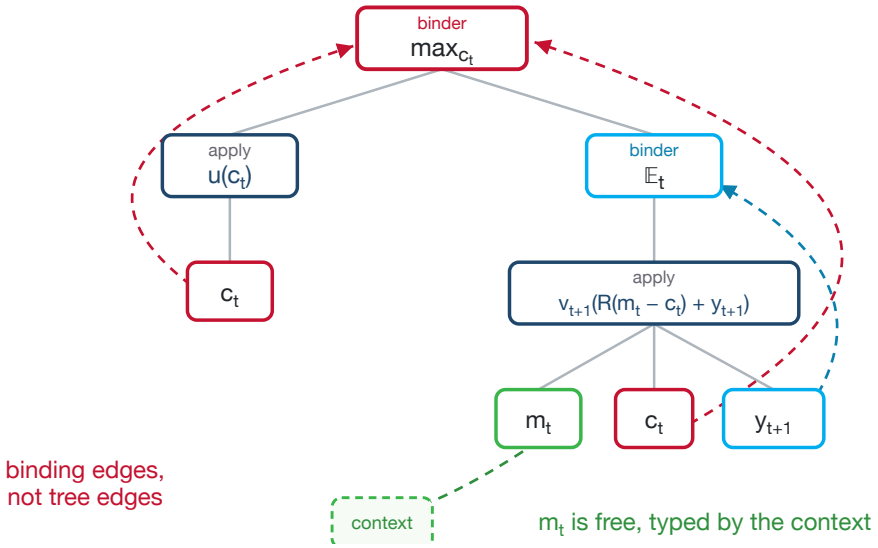
What the tree records. The tree records only which symbol is applied to which, node by node.

What the tree omits. The tree records nothing about what `v`, `c` or `w` are, which variable is bound where, or whether `v` is a function.

The tree is Python's abstract syntax tree, `ast.parse("v = u(c) + beta * w")`.

From the syntax tree to the semantic graph

In the model, v is not a number obtained by adding βw to $u(c)$. That $v_t = \mathbb{T}\, v_{t+1}$ sends a function to a function appears only once the binders, types and declarations on the right are attached to the tree, the dotted lines in the figure.



binders: $\max_{c_t}$ binds c_t, $\quad \mathbb{E}_t$ binds y_{t+1},
$v_t(m_t) = \cdots \quad \forall m_t \in \mathbb{R}_{++}$ binds m_t

fixed from outside: $u : \mathbb{R}_{++} \to \mathbb{R}, \quad \beta,\ R \in \mathbb{R},$
ν_y the law of y_{t+1}, on $\mathbb{R}_{++}$

obtained: $v_{t+1}\big(R(m_t - c_t) + y_{t+1}\big) : \mathbb{R},$
$\mathbb{T} : (\mathbb{R}_{++} \to \mathbb{R}) \to (\mathbb{R}_{++} \to \mathbb{R})$

The law ν_y is the probability distribution of the income shock. A semantic ontology states the quantifier over m_t on the left-hand side, the binder conventions, the types and the declarations of u and ν_y, none of which the tree records.

Bellman-SYM, Project Bellman's model language, parses a file into such a tree, builds the semantic graph from it by a checked interpretation, which the calculus calls Y (upsilon), and discards the tree (Bellman calculus, Introduction §2; Bellman-SYM, Stages §5).

What the expectation integrates

The paper's Bellman equation gives $\mathbb{E}$ the date t as its subscript rather than the variable it integrates. Because the tree specifies neither that variable nor the type of $\mathbb{E}_t$, each box below puts the paper's expression to the left of "read as" and a reading that the tree does not exclude to the right.

The shock leaves the expectation.

$$\mathbb{E}_t\, v\big(R(m-c)+y'\big) \qquad \text{read as} \qquad v\big(R(m-c)+\mathbb{E}_t\, y'\big) \quad \text{or} \quad \mathbb{E}_t\big[v\big(R(m-c)\big)\big]+y'.$$

Since Jensen's inequality gives $\mathbb{E}_t\, v(R(m-c)+y') \le v(R(m-c)+\mathbb{E}_t\, y')$ for concave v, the first reading overstates the continuation value. Because it replaces the income risk by its mean, it also removes the precautionary motive, the saving held against that risk (Kimball 1990). The second reading treats y' as known before it is drawn.

The expectation moved through the maximization.

$$\max_c \big\{u(c)+\beta\, \mathbb{E}_t\, v_{t+1}(\cdot)\big\} \qquad \text{read as} \qquad \mathbb{E}_t \max_c \big\{u(c)+\beta\, v_{t+1}(\cdot)\big\}.$$

Since $\mathbb{E}\max \ge \max\mathbb{E}$, the reading on the right overstates the value and lets consumption respond to income that has not yet been drawn.

Jensen's inequality and the precautionary motive: Kimball (1990), "Precautionary saving in the small and in the large", *Econometrica* 58(1), 53–73.

A number for a function, and a function on the wrong space

A number where a function belongs.

$v_t = \mathbb{T}\, v_{t+1}, \quad v_t : \mathbb{R}_{++} \to \mathbb{R}$ read as $v_t(\hat{m}_t)$ alone, at $\hat{m}_t = R\,\hat{k}_t + \hat{y}_t,$ one simulated household's resources.

The backward step then returns one number and one consumption $c_t(\hat{m}_t)$, although $\mathbb{T}$ needs v_{t+1} at every $R(\hat{m}_t - c) + y'$. Under this reading the reader has simulated a path where the model asked for a function.

A function on the wrong space.

$\mathbb{E}_t\, v_{t+1}(Ra + y')$ on savings $a = m - c,$ the paper's $k_{t+1},$ $a \in [0, \infty)$ read as v_{t+1} itself, on resources $m \in (0, \infty)$.

The borrowing limit acts at $a = 0$, a point at which a function of resources is not defined, so that the two functions have different domains. A reading that confuses them puts the boundary at a point where the constraint does not act.

The discount and the expectation applied twice, and the recursion as a fixed point

The discount and the expectation applied by both steps once $\mathbb{T}$ is split.

$$v_t = \max_c \left\{ u(c) + \beta \, \mathbb{E}_t \, v_{t+1}(\cdot) \right\} \qquad \text{read as} \qquad v_t = \max_c \left\{ u(c) + \beta^2 \, \mathbb{E}_{y'} \mathbb{E}_{y''} \, v_{t+1}(\cdot) \right\}.$$

Since the Bellman equation writes β and $\mathbb{E}_t$ once each, an implementation that splits $\mathbb{T}$ into the choice and the expectation must decide which step applies β and which applies $\mathbb{E}_t$. An implementation that applies β and $\mathbb{E}_t$ in each of the two steps makes the household discount at β^2 and draw income twice.

Dated value functions read as one stationary function.

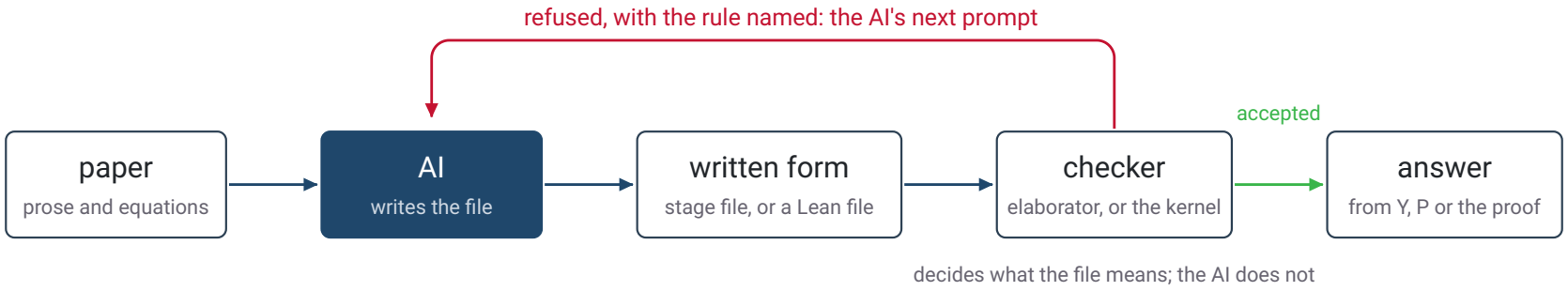
$$v_t = \mathbb{T} \, v_{t+1}, \;\; v_{t+1} \text{ an input} \qquad \text{read as} \qquad v = \mathbb{T} \, v, \;\; \text{a fixed point.}$$

A stationarity condition that the statement never makes replaces the backward recursion from a terminal value. Once the terminal value has disappeared from the problem, the last period's consumption is chosen as if another period followed.

Appendix

How an AI works with a semantic ontology

The AI writes against the ontology



Autoformalization. The AI translates the paper into the written form, and the checker accepts or refuses the result (Wu et al. 2022; AlphaProof, Hubert et al. 2025, which learned by reinforcement with Lean's kernel as the verifier).

Refuse and repair. The checker's refusal, with its rule named, is the AI's next prompt, as a proof assistant's error message is in Baldur (First et al. 2023). The elaborator's rules `shock-reintroduced` and `field-perch` are messages of this kind.

Constrained generation. The AI's output is restricted at decoding time to the grammar of the written form, so that only its meaning remains to be checked (Poesia et al. 2022; Geng et al. 2023).

Compute, then answer. From the accepted file the ontology's programs compute, Y the object and P the numbers, or the kernel the proof, and the AI's answer to the user is built from that output rather than from its own guess.

Round trip. A second program writes the accepted object back into prose, and a judge compares that prose with the paper (Li et al. 2024; Allamanis et al. 2024). Matsya's architect mode runs this loop with the elaborator and two judges.

Wu, Jiang, Li, Rabe, Szegedy, Jamnik and Jaeger (2022), "Autoformalization with large language models", *NeurIPS 2022*, 32353–32368 · Hubert, Mehta, Sartran et al. (2025), *Nature* 651, 607–613 · First, Rabe, Ringer and Brun (2023), "Baldur: whole-proof generation and repair with large language models", *ESEC/FSE 2023*, 1229–1241 · Poesia et al. (2022), "Synchromesh", *ICLR 2022*; Geng, Josifoski, Peyrard and West (2023), "Grammar-constrained decoding for structured NLP tasks without finetuning", *EMNLP 2023*, 10932–10952 · Li, Wu, Li, Wei, Yang, Zhang and Ma (2024), "Autoformalize mathematical statements by symbolic equivalence and semantic consistency", *NeurIPS 2024*, 53598–53625; Allamanis, Panthaplackel and Yin (2024), "Unsupervised evaluation of code LLMs with round-trip correctness", *ICML 2024*, PMLR 235, 1050–1066 · Matsya's architect mode: bellman-ddsl, terminology, 9 October 2026.

The AI reads from the ontology

Retrieval over typed objects. The AI answers from terms, definitions and relations retrieved from the ontology, each with a stated meaning, rather than from passages of text (Pan et al. 2024; Edge et al. 2024). Matsya's retrieval over the project's typed documents is this.

Training on generated pairs. The ontology generates pairs of a file and its prose, or the comparator labels two files as one object or as two, and the AI is trained on the pairs. AlphaProof's roughly one million natural-language problems, auto-formalized into about 80 million formal statements, are the largest instance, and the project's comparator module, `meaning_map_training` , is named for this use.

What separates the two groups. In the five uses of the previous slide the checker, not the AI, decides what the file means, so the AI's reading can be inspected. In these two the AI's reading improves but stays statistical, which is why they do not replace the checker.

Pan, Luo, Wang, Chen, Wang and Wu (2024), "Unifying large language models and knowledge graphs: a roadmap", *IEEE Transactions on Knowledge and Data Engineering* 36(7), 3580–3599 · Edge et al. (2024), "From local to global: a graph RAG approach to query-focused summarization", arXiv:2404.16130 · Hubert, Mehta, Sartran et al. (2025), *Nature* 651, 607–613, on the auto-formalized training problems · `meaning_map_training.compare` : bellman-ddsl.